Python + Linux + Shell Scripting

300 Interview Questions & Answers — including coding and practical command questions

Designed for interview revision. Answers are intentionally concise, with practical examples and code for common coding questions.

Section	Questions
Python	134
Python Coding	38
Linux	101
Linux Practical	20
Shell Scripting	7

Python

1. What is Python?

Answer: Python is a high-level, interpreted, general-purpose programming language known for readable syntax and a large standard library.

2. Why is Python popular?

Answer: It has simple syntax, extensive libraries, cross-platform support, and is useful for automation, testing, web development, data work, and scripting.

3. Is Python compiled or interpreted?

Answer: Python source is compiled to bytecode and then executed by the Python virtual machine. It is commonly described as an interpreted language.

4. What is PEP 8?

Answer: PEP 8 is the main Python style guide. It gives conventions for formatting, naming, imports, whitespace, and readable code.

5. What is a variable in Python?

Answer: A variable is a name bound to an object. Python variables do not require an explicit type declaration.

6. Is Python dynamically typed?

Answer: Yes. A variable can refer to objects of different types at different times.

7. What is strong typing?

Answer: Python does not silently combine unrelated types in many operations; for example, adding a string and integer raises TypeError.

8. Name common Python built-in data types.

Answer: int, float, complex, bool, str, list, tuple, set, dict, bytes, bytearray, range, and NoneType.

9. What is None?

Answer: None is Python's singleton object representing absence of a value or no meaningful result.

10. What is type() used for?

Answer: type() returns the type of an object.

11. What is isinstance() used for?

Answer: isinstance() checks whether an object is an instance of a specified class or a subclass.

12. Difference between == and is?

Answer: == compares values for equality; is checks whether two references point to the same object.

13. What is mutable?

Answer: A mutable object can be changed after creation. Lists, dictionaries, and sets are mutable.

14. What is immutable?

Answer: An immutable object cannot be changed after creation. Integers, strings, tuples, and bytes are immutable.

15. Are tuples always immutable?

Answer: The tuple structure is immutable, but a tuple can contain mutable objects whose contents can still change.

16. What is a list?

Answer: A list is an ordered, mutable collection that can contain duplicate values and mixed types.

17. What is a tuple?

Answer: A tuple is an ordered, immutable sequence that can contain duplicate values.

18. What is a set?

Answer: A set is an unordered collection of unique hashable elements.

19. What is a dictionary?

Answer: A dictionary stores key-value pairs. Keys must be hashable.

20. Can a list be a dictionary key?

Answer: No. A list is mutable and unhashable, so it cannot be a dictionary key.

21. What is slicing?

Answer: Slicing extracts part of a sequence using start, stop, and optional step: sequence[start:stop:step].

22. What does [::-1] do?

Answer: It returns a reversed copy of a sequence such as a string or list.

23. What is a list comprehension?

Answer: It is a compact syntax for creating a list from an iterable, optionally with a condition.

24. What is a dictionary comprehension?

Answer: It is a compact syntax for constructing a dictionary from an iterable.

25. What is a generator expression?

Answer: It creates a lazy iterator using parentheses, producing values when requested instead of building the whole collection.

26. What is an iterator?

Answer: An iterator is an object that implements __iter__() and __next__() and yields values one at a time.

27. What is an iterable?

Answer: An iterable is an object that can return its elements one at a time, usually through __iter__().

28. Iterator vs iterable?

Answer: An iterable can produce an iterator; an iterator maintains iteration state and provides __next__().

29. What is a function?

Answer: A function is a reusable block of code defined with def.

30. What is a parameter?

Answer: A parameter is a variable in a function definition that receives an argument.

31. What is an argument?

Answer: An argument is a value supplied when calling a function.

32. What are default arguments?

Answer: They are parameter values used when the caller does not provide that argument.

33. What are keyword arguments?

Answer: They pass values by parameter name, such as func(name='Sam').

*34. What is *args?*

Answer: *args collects extra positional arguments into a tuple.

*35. What is **kwargs?*

Answer: **kwargs collects extra keyword arguments into a dictionary.

36. What is return?

Answer: return exits a function and optionally sends a value back to the caller.

37. What happens if a function has no return?

Answer: It implicitly returns None.

38. What is a lambda?

Answer: A lambda is a small anonymous function written as lambda parameters: expression.

39. What is recursion?

Answer: Recursion is when a function calls itself, normally with a base case to stop the calls.

40. What is a module?

Answer: A module is a Python file containing code such as functions, classes, and variables.

41. What is a package?

Answer: A package is a directory structure used to organize Python modules; modern Python packages can be namespace packages without __init__.py.

42. What is import?

Answer: import loads a module or names from a module so their functionality can be used.

43. What is __name__ == '__main__'?

Answer: It makes code run as a script entry point only when the file is executed directly, not when imported.

44. What is pip?

Answer: pip is the standard Python package installer used to install and manage packages from package indexes.

45. What is a virtual environment?

Answer: It is an isolated Python environment with its own installed packages, helping prevent dependency conflicts.

46. How do you create a virtual environment?

Answer: Use python -m venv venv and activate it using the platform-specific activation command.

47. What is an exception?

Answer: An exception is an event raised during execution that interrupts normal control flow.

48. What is try/except?

Answer: It catches specified exceptions and lets the program handle errors without terminating unexpectedly.

49. What is finally?

Answer: finally contains code that runs after try/except processing, whether an exception occurred or not.

50. What is else in try/except?

Answer: The else block runs when the try block completes without raising an exception.

51. What is raise?

Answer: raise explicitly raises an exception.

52. What is a custom exception?

Answer: A user-defined exception is normally created by subclassing Exception.

53. What is file handling?

Answer: It means opening, reading, writing, and closing files using Python file objects.

54. Why use with open()?

Answer: The with statement automatically closes the file even if an exception occurs.

55. What are common file modes?

Answer: r read, w write/truncate, a append, x create, b binary, and + update/read-write.

56. What is JSON?

Answer: JSON is a text format for structured data. Python provides the json module for encoding and decoding it.

57. What is serialization?

Answer: Serialization converts an object or data structure into a storable/transmittable representation.

58. What is deserialization?

Answer: Deserialization reconstructs data from a serialized representation.

59. What is OOP?

Answer: Object-oriented programming organizes software around objects containing data and behavior.

60. What is a class?

Answer: A class is a blueprint defining attributes and methods for objects.

61. What is an object?

Answer: An object is an instance of a class.

62. What is __init__?

Answer: __init__ is an initializer method called when a new instance is created.

63. What is self?

Answer: self refers to the current instance inside an instance method.

64. What is inheritance?

Answer: Inheritance lets a class derive behavior and attributes from another class.

65. What is polymorphism?

Answer: Polymorphism allows different objects to provide a common interface with type-specific behavior.

66. What is encapsulation?

Answer: Encapsulation groups data and methods and controls how implementation details are accessed.

67. What is abstraction?

Answer: Abstraction exposes essential behavior while hiding unnecessary implementation details.

68. What is method overriding?

Answer: A subclass provides its own implementation of a method inherited from a parent class.

69. What is super()?

Answer: super() gives access to parent-class behavior, commonly used to call inherited methods or initializers.

70. What is a class variable?

Answer: A class variable is shared by instances through the class unless an instance shadows it.

71. What is an instance variable?

Answer: An instance variable belongs to a particular object, usually stored through self.

72. What is a static method?

Answer: A @staticmethod does not receive self or cls automatically and behaves like a function stored on a class.

73. What is a class method?

Answer: A @classmethod receives cls and can operate on class-level state or construct instances.

74. What is a decorator?

Answer: A decorator wraps a function or class to add or modify behavior without changing its core source.

75. What is a closure?

Answer: A closure is a nested function that remembers variables from its enclosing scope after the outer function returns.

76. What is LEGB?

Answer: LEGB describes name lookup order: Local, Enclosing, Global, Built-in.

77. What is global?

Answer: global declares that assignments inside a function refer to a module-level variable.

78. What is nonlocal?

Answer: nonlocal lets a nested function assign to a variable in an enclosing non-global scope.

79. What is shallow copy?

Answer: A shallow copy creates a new outer object but keeps references to nested objects.

80. What is deep copy?

Answer: A deep copy recursively copies nested objects using the copy module.

81. What is garbage collection?

Answer: Python manages memory automatically through reference counting and a cyclic garbage collector in CPython.

82. What is reference counting?

Answer: In CPython, objects track references; when the count reaches zero, the object can usually be deallocated immediately.

83. What is the GIL?

Answer: In standard CPython, the Global Interpreter Lock allows only one thread at a time to execute Python bytecode in a process, affecting CPU-bound threading.

84. Threading vs multiprocessing?

Answer: Threading uses threads in one process and is useful for many I/O-bound tasks; multiprocessing uses separate processes and can use multiple CPU cores for CPU-bound work.

85. What is asyncio?

Answer: asyncio provides asynchronous programming with an event loop and async/await, useful for high-concurrency I/O.

86. What is a regular expression?

Answer: A regular expression is a pattern used to search, match, split, or replace text. Python provides re.

87. What is re.match()?

Answer: It attempts a match at the beginning of a string.

88. What is re.search()?

Answer: It searches for a match anywhere in a string.

89. What is re.findall()?

Answer: It returns all non-overlapping matches, usually as a list of strings or tuples.

90. What is environment variable?

Answer: It is a key-value setting supplied by the operating system or process environment, accessible in Python through os.environ.

91. What is subprocess?

Answer: The subprocess module lets Python create and communicate with external processes.

92. Why avoid shell=True when unnecessary?

Answer: It can introduce shell injection risks when untrusted input is included and is often unnecessary for simple commands.

93. What is logging?

Answer: The logging module provides structured application messages with levels such as DEBUG, INFO, WARNING, ERROR, and CRITICAL.

94. print vs logging?

Answer: print is simple console output; logging supports levels, handlers, formatting, files, and configurable destinations.

95. What is unit testing?

Answer: Unit testing checks small pieces of code independently, commonly using unittest or pytest.

96. What is mocking?

Answer: Mocking replaces a dependency with a controlled test double so behavior can be tested in isolation.

97. What is an assertion?

Answer: An assertion checks an expected condition and raises an error if it is false.

98. What is type hinting?

Answer: Type hints annotate expected types for readability, static analysis, and tooling; Python generally does not enforce them at runtime.

99. What is a docstring?

Answer: A docstring is a string literal documenting a module, class, or function.

100. What is pass?

Answer: pass is a no-operation statement used where syntactic code is required.

101. break vs continue?

Answer: break exits the loop; continue skips the rest of the current iteration and proceeds to the next.

102. What does enumerate() do?

Answer: It returns an iterable of index-value pairs while looping over a sequence.

103. What does zip() do?

Answer: zip combines elements from multiple iterables position by position.

104. What does sorted() do?

Answer: sorted() returns a new sorted list without modifying the original iterable.

105. sort() vs sorted()?

Answer: list.sort() sorts a list in place and returns None; sorted() returns a new list.

106. What does map() do?

Answer: map applies a function to each item of an iterable and returns an iterator.

107. What does filter() do?

Answer: filter keeps items for which a predicate is true and returns an iterator.

108. What is any()?

Answer: any() returns True if at least one element of an iterable is truthy.

109. What is all()?

Answer: all() returns True only if every element of an iterable is truthy.

110. What is truthy/falsy?

Answer: Objects have Boolean truth values. Examples of falsy values include False, None, 0, 0.0, '', empty collections.

111. What is unpacking?

Answer: Unpacking assigns elements of an iterable to multiple variables, such as a, b = [1, 2].

112. What is extended unpacking?

Answer: Syntax such as a, *middle, b = values captures remaining elements in a list.

113. What is hashable?

Answer: A hashable object has a stable hash value and can be used as a dictionary key or set element.

114. What is an f-string?

Answer: An f-string embeds expressions directly in strings, for example f'Value={x}'.

115. What is strip()?

Answer: strip() removes leading and trailing characters, whitespace by default.

116. split() vs join()?

Answer: split() converts a string into a list using a separator; join() combines strings using a separator.

117. What is replace()?

Answer: replace(old, new) returns a string with occurrences of old replaced by new.

118. How do you convert string to integer?

Answer: Use int(), for example int('123').

119. How do you handle invalid integer conversion?

Answer: Use try/except ValueError around int().

120. What is range()?

Answer: range() represents an arithmetic progression of integers and is commonly used for loops.

121. What is the difference between / and //?

Answer: / performs true division and returns a float; // performs floor division.

122. What does % do?

Answer: % returns the remainder of division.

*123. What does ** do?*

Answer: ** performs exponentiation.

124. How do you remove duplicates from a list?

Answer: Use a set when order is unimportant, or use an order-preserving approach such as list(dict.fromkeys(values)).

125. How do you count list elements?

Answer: Use len(list) for total elements or collections.Counter for frequency counts.

126. How do you check a key in a dictionary?

Answer: Use key in dictionary.

127. How do you merge dictionaries?

Answer: Use d1 | d2 in supported Python versions, or d1.update(d2) when mutation is acceptable.

128. What is defaultdict?

Answer: collections.defaultdict creates missing values using a supplied factory function.

129. What is Counter?

Answer: collections.Counter counts hashable objects and provides convenient frequency operations.

130. What is deque?

Answer: collections.deque is a double-ended queue optimized for appending and popping at both ends.

131. What is time complexity?

Answer: It describes how resource usage grows with input size, such as O(1), O(n), or O(n log n).

132. What is a Pythonic approach?

Answer: It is code that follows Python conventions and uses clear, idiomatic language features appropriately.

133. How can Python code be made faster?

Answer: Use appropriate algorithms/data structures, avoid unnecessary work, profile first, and consider vectorized/native libraries or multiprocessing for suitable workloads.

134. What is profiling?

Answer: Profiling measures where a program spends time or resources so optimization can target real bottlenecks.

Python Coding

135. Reverse a string.

Answer: Use slicing for a concise solution.

```
s = "hello"
print(s[::-1])
```

136. Check whether a string is a palindrome.

Answer: Compare the string with its reverse.

```
s = "madam"
print(s == s[::-1])
```

137. Find the largest number in a list.

Answer: Use max() or a loop if demonstrating logic.

```
nums = [10, 4, 25, 7]
print(max(nums))
```

138. Find the smallest number in a list.

Answer: Use min().

```
nums = [10, 4, 25, 7]
print(min(nums))
```

139. Find the sum of a list.

Answer: Use sum().

```
nums = [1, 2, 3, 4]
print(sum(nums))
```

140. Count even numbers in a list.

Answer: Check each number with modulo 2.

```
nums = [1, 2, 4, 7, 8]
count = sum(1 for x in nums if x % 2 == 0)
```

```
print(count)
```

141. Print numbers from 1 to 10.

Answer: Use range(1, 11).

```
for i in range(1, 11):
    print(i)
```

142. Print a multiplication table.

Answer: Loop from 1 to 10 and multiply by the given number.

```
n = 5
for i in range(1, 11):
    print(n, "x", i, "=", n*i)
```

143. Calculate factorial.

Answer: Multiply integers from 1 through n.

```
n = 5
fact = 1
for i in range(1, n + 1):
    fact *= i
print(fact)
```

144. Generate Fibonacci numbers.

Answer: Keep the previous two values and update them each iteration.

```
a, b = 0, 1
for _ in range(10):
    print(a)
    a, b = b, a + b
```

145. Check whether a number is prime.

Answer: Test divisibility from 2 through the square root of n.

```
n = 29
is_prime = n >= 2
for i in range(2, int(n**0.5) + 1):
    if n % i == 0:
        is_prime = False
        break
print(is_prime)
```

146. Count vowels in a string.

Answer: Convert to a consistent case and count characters in a vowel set.

```
s = "Embedded Python"
count = sum(ch.lower() in "aeiou" for ch in s)
print(count)
```

147. Count words in a sentence.

Answer: Split on whitespace and count the resulting words.

```
s = "Python is easy to learn"
print(len(s.split()))
```

148. Find duplicate values in a list.

Answer: Track seen values and report values encountered again.

```
nums = [1, 2, 2, 3, 4, 4]
seen = set()
dup = set()
for x in nums:
    if x in seen:
        dup.add(x)
    seen.add(x)
print(dup)
```

149. Remove duplicates while preserving order.

Answer: dict.fromkeys preserves insertion order in modern Python.

```
nums = [3, 1, 3, 2, 1]
print(list(dict.fromkeys(nums)))
```

150. Count frequency of each character.

Answer: Use a dictionary or Counter.

```
from collections import Counter
s = "banana"
print(Counter(s))
```

151. Find the second largest distinct number.

Answer: Sort unique values and select the second-last item.

```
nums = [10, 20, 20, 5, 30]
u = sorted(set(nums))
print(u[-2])
```

152. Swap two variables.

Answer: Python supports multiple assignment.

```
a, b = 10, 20
a, b = b, a
print(a, b)
```

153. Check whether two strings are anagrams.

Answer: Normalize and compare sorted characters.

```
a = "listen"
b = "silent"
print(sorted(a) == sorted(b))
```

154. Flatten a one-level nested list.

Answer: Use a nested list comprehension.

```
data = [[1, 2], [3, 4], [5]]
flat = [x for row in data for x in row]
print(flat)
```

155. Find common elements in two lists.

Answer: Use set intersection when order is not required.

```
a = [1, 2, 3, 4]
b = [3, 4, 5]
print(set(a) & set(b))
```

156. Read a text file and count lines.

Answer: Use a with block and iterate over the file.

```
count = 0
with open("data.txt", "r") as f:
    for _ in f:
        count += 1
print(count)
```

157. Write text to a file.

Answer: Open in write mode and write the string.

```
with open("out.txt", "w") as f:
    f.write("Hello\n")
```

158. Read a file safely.

Answer: Use with so the file closes automatically.

```
with open("data.txt", "r") as f:
    text = f.read()
print(text)
```

159. Handle division by zero.

Answer: Catch ZeroDivisionError.

```
try:
```

```
    print(10 / 0)
except ZeroDivisionError:
    print("Cannot divide by zero")
```

160. Find the first non-repeating character.

Answer: Count characters first, then scan in original order.

```
from collections import Counter
s = "swiss"
c = Counter(s)
print(next((ch for ch in s if c[ch] == 1), None))
```

161. Sort a list of tuples by the second element.

Answer: Use sorted with a key function.

```
data = [("a", 3), ("b", 1), ("c", 2)]
print(sorted(data, key=lambda x: x[1]))
```

162. Extract numbers greater than 10.

Answer: Use a list comprehension with a condition.

```
nums = [5, 12, 8, 20]
print([x for x in nums if x > 10])
```

163. Check if a list is sorted.

Answer: Compare it with its sorted version.

```
nums = [1, 2, 3, 4]
print(nums == sorted(nums))
```

164. Find maximum without max().

Answer: Maintain a running maximum.

```
nums = [4, 9, 2, 7]
big = nums[0]
for x in nums[1:]:
    if x > big:
        big = x
print(big)
```

165. Find factorial using recursion.

Answer: Use a base case at 0 or 1.

```
def fact(n):
    if n <= 1:
        return 1
    return n * fact(n-1)
print(fact(5))
```

166. Check Armstrong number.

Answer: Sum each digit raised to the number of digits and compare with the original.

```
n = 153
digits = str(n)
print(sum(int(d)**len(digits) for d in digits) == n)
```

167. Reverse a list without reverse().

Answer: Use slicing or swap elements manually.

```
nums = [1, 2, 3, 4]
print(nums[::-1])
```

168. Find missing number from 1..n.

Answer: Use the arithmetic sum formula and subtract the actual sum.

```
nums = [1, 2, 4, 5]
n = 5
missing = n*(n+1)//2 - sum(nums)
print(missing)
```

169. Merge two dictionaries.

Answer: Use dictionary unpacking or the | operator.

```
a = {"x": 1}
b = {"y": 2}
print({**a, **b})
```

170. Run an external Linux command from Python.

Answer: Use subprocess.run and pass arguments as a list when possible.

```
import subprocess
result = subprocess.run(["ls", "-l"], capture_output=True, text=True)
print(result.stdout)
```

171. Accept a command-line argument.

Answer: Use sys.argv or argparse.

```
import sys
print(sys.argv[1])
```

172. Parse command-line arguments cleanly.

Answer: argparse provides named options, help text, and validation.

```
import argparse
p = argparse.ArgumentParser()
p.add_argument("--name", required=True)
args = p.parse_args()
print(args.name)
```

Linux

173. What is Linux?

Answer: Linux is an open-source Unix-like kernel; distributions combine the kernel with user-space tools and software.

174. What is a Linux distribution?

Answer: A distribution packages the Linux kernel with utilities, package management, libraries, and applications, such as RHEL, Ubuntu, or Debian.

175. What is the kernel?

Answer: The kernel manages hardware resources, memory, processes, devices, networking, and system calls.

176. What is a shell?

Answer: A shell is a command interpreter that lets users run commands and scripts. Bash is a common shell.

177. What is Bash?

Answer: Bash is the Bourne Again Shell, a widely used command-line shell and scripting language.

178. pwd?

Answer: Prints the current working directory.

179. ls?

Answer: Lists directory contents.

180. ls -l?

Answer: Shows a long listing with permissions, ownership, size, and timestamps.

181. ls -a?

Answer: Shows hidden entries whose names normally begin with a dot.

182. cd?

Answer: Changes the current working directory.

183. cd ..?

Answer: Moves to the parent directory.

184. mkdir?

Answer: Creates directories.

185. mkdir -p?

Answer: Creates parent directories as needed without failing when they already exist.

186. touch?

Answer: Creates an empty file if it does not exist or updates timestamps if it does.

187. cp?

Answer: Copies files or directories when appropriate options are supplied.

188. mv?

Answer: Moves or renames files and directories.

189. rm?

Answer: Removes files; recursive options are needed for directories.

190. cat?

Answer: Displays or concatenates file contents.

191. less?

Answer: Views a file interactively with scrolling and search.

192. head?

Answer: Displays the beginning of a file or input.

193. tail?

Answer: Displays the end of a file or input.

194. tail -f?

Answer: Follows a file as new data is appended, commonly used for live logs.

195. grep?

Answer: Searches input or files for lines matching a pattern.

196. grep -i?

Answer: Performs case-insensitive matching.

197. grep -r?

Answer: Recursively searches directories.

198. find?

Answer: Searches for files and directories based on criteria such as name, type, size, or time.

199. locate?

Answer: Searches a prebuilt file-name database; results can be stale until the database is updated.

200. which?

Answer: Shows the path of an executable found through PATH.

201. whereis?

Answer: Locates binaries, source, and manual pages associated with a command.

202. man?

Answer: Displays manual pages for commands and system interfaces.

203. echo?

Answer: Prints arguments or variable values.

204. whoami?

Answer: Prints the effective current username.

205. id?

Answer: Displays user and group identity information.

206. uname -a?

Answer: Displays system and kernel information.

207. hostname?

Answer: Displays or, with appropriate options, changes the system hostname.

208. date?

Answer: Displays the current date and time.

209. clear?

Answer: Clears the terminal display.

210. history?

Answer: Shows previously executed shell commands.

211. alias?

Answer: Defines a shortcut name for a command or command sequence.

212. What is an absolute path?

Answer: A path beginning at the filesystem root, such as /var/log/messages.

213. What is a relative path?

Answer: A path interpreted relative to the current working directory.

214. What is /?

Answer: The root of the Linux filesystem hierarchy.

215. What is /home?

Answer: A conventional location for users' home directories.

216. What is /etc?

Answer: A conventional directory for system and application configuration files.

217. What is /var?

Answer: Stores variable data such as logs, caches, spools, and application state.

218. What is /tmp?

Answer: A conventional location for temporary files.

219. What is /usr?

Answer: Contains many user-space programs, libraries, documentation, and related resources.

220. What is /dev?

Answer: Contains device files representing devices exposed to user space.

221. What is /proc?

Answer: A virtual filesystem exposing process and kernel information.

222. What is /sys?

Answer: A virtual filesystem exposing kernel/device information and interfaces.

223. What is a file descriptor?

Answer: A small integer used by a process to refer to an open file or I/O resource.

224. Standard input/output/error?

Answer: stdin is file descriptor 0, stdout is 1, and stderr is 2.

225. What is a pipe?

Answer: The | operator sends stdout of one command to stdin of another.

226. What is redirection >?

Answer: It sends stdout to a file, replacing/truncating the file if it already exists.

227. What is >>?

Answer: It appends stdout to a file.

228. What is 2>?

Answer: It redirects standard error to a file.

229. What is &>?

Answer: In Bash, it redirects stdout and stderr together.

230. What is chmod?

Answer: Changes file permission bits.

231. What is chown?

Answer: Changes file owner and optionally group.

232. What is chgrp?

Answer: Changes a file's group ownership.

233. What does rwx mean?

Answer: Read, write, and execute permissions.

234. What does chmod 755 mean?

Answer: Owner gets rwx; group and others get r-x.

235. What does chmod 644 mean?

Answer: Owner gets rw-; group and others get r--.

236. What is umask?

Answer: It specifies permission bits that are masked when new files/directories are created.

237. What is a process?

Answer: A running instance of a program with its own execution state and resources.

238. ps?

Answer: Displays information about processes.

239. ps aux?

Answer: A common form showing processes for all users with detailed information.

240. top?

Answer: Provides a dynamic view of processes and resource usage.

241. kill?

Answer: Sends a signal to a process; it does not necessarily terminate it immediately.

242. kill -9?

Answer: Sends SIGKILL, which cannot be caught or ignored; use it carefully.

243. What is a signal?

Answer: A software notification delivered to a process to indicate an event or request action.

244. What is PID?

Answer: Process ID, a unique identifier for a running process on the system.

245. What is PPID?

Answer: Parent Process ID, identifying the process that created a process.

246. What is a daemon?

Answer: A background service process, often started at boot or by a service manager.

247. systemctl?

Answer: Controls and queries systemd-managed services and system state.

248. journalctl?

Answer: Queries logs collected by systemd-journald.

249. df?

Answer: Reports filesystem disk-space usage.

250. du?

Answer: Estimates disk usage of files and directories.

251. free?

Answer: Displays memory and swap usage.

252. uptime?

Answer: Shows how long the system has been running and basic load averages.

253. ssh?

Answer: Secure Shell provides encrypted remote login and command execution.

254. scp?

Answer: Securely copies files over SSH.

255. tar?

Answer: Archives files and directories and can optionally compress them with supported tools.

256. gzip?

Answer: Compresses data using the gzip format.

257. tar -czf?

Answer: Creates a gzip-compressed tar archive.

258. tar -xzf?

Answer: Extracts a gzip-compressed tar archive.

259. What is cron?

Answer: A scheduler for recurring jobs, traditionally configured through crontab.

260. crontab?

Answer: Stores scheduled cron jobs for a user.

261. What is PATH?

Answer: An environment variable containing directories searched for executable commands.

262. What is an environment variable?

Answer: A variable inherited by processes from their environment, commonly used for configuration.

263. export?

Answer: Marks a shell variable for export to child processes.

264. What is sudo?

Answer: Runs a command with another user's privileges, commonly root, subject to policy.

265. root user?

Answer: The privileged administrative account on Unix-like systems.

266. What is package management?

Answer: Installing, updating, removing, and tracking software packages using a distribution's package manager.

267. RHEL package manager?

Answer: Modern RHEL systems commonly use dnf; yum is a compatibility command in many releases.

268. What is RPM?

Answer: RPM is a package format and package-management technology used by RHEL-family distributions.

269. What is an inode?

Answer: A filesystem data structure containing metadata about a file, such as ownership, permissions, and block references; the filename is stored in a directory entry.

270. Hard link vs symbolic link?

Answer: A hard link refers to the same inode; a symbolic link stores a path to another file or directory.

271. ln -s?

Answer: Creates a symbolic link.

272. What is mount?

Answer: Attaches a filesystem to a directory in the Linux filesystem tree.

273. What is NFS?

Answer: Network File System allows directories/filesystems to be shared over a network.

Linux Practical

274. Find all .log files under /var/log.

Answer: Use find with a name pattern.

```
find /var/log -type f -name "*.log"
```

275. Search for ERROR in a log recursively.

Answer: Use recursive grep.

```
grep -R "ERROR" /var/log
```

276. Show the last 50 lines of a file.

Answer: Use tail -n.

```
tail -n 50 app.log
```

277. Follow a log file live.

Answer: Use tail -f.

```
tail -f app.log
```

278. Count lines containing ERROR.

Answer: Pipe grep into wc -l.

```
grep -c "ERROR" app.log
```

279. Count total lines in a file.

Answer: Use wc -l.

```
wc -l app.log
```

280. Find the largest files in a directory.

Answer: Combine du, sort, and head; exact output varies by platform.

```
du -ah /path | sort -hr | head -10
```

281. Check disk usage of mounted filesystems.

Answer: Use df with human-readable output.

```
df -h
```

282. Check memory usage.

Answer: Use free with human-readable output.

```
free -h
```

283. Find a process by name.

Answer: Use pgrep or ps with grep.

```
pgrep -af python
```

284. Kill a process by PID.

Answer: Send the default termination signal first.

```
kill 1234
```

285. Change a script to executable.

Answer: Add execute permission.

```
chmod +x script.sh
```

286. Run a script from the current directory.

Answer: Use ./ when the current directory is not necessarily in PATH.

```
./script.sh
```

287. Redirect stdout and stderr to one file in Bash.

Answer: Use &> or equivalent redirections.

```
./script.sh &> output.log
```

288. Append command output to a file.

Answer: Use >>.

```
date >> dates.log
```

289. Pipe output through grep.

Answer: Connect stdout of the first command to stdin of grep.

```
ps aux | grep python
```

290. Create a gzip tar archive.

Answer: Use tar with c, z, and f options.

```
tar -czf backup.tar.gz project/
```

291. Extract a gzip tar archive.

Answer: Use tar with x, z, and f.

```
tar -xzf backup.tar.gz
```

292. Find files modified in the last day.

Answer: Use find with -mtime.

```
find /path -type f -mtime -1
```

293. Securely copy a file to another Linux host.

Answer: Use scp over SSH.

```
scp report.txt user@host:/tmp/
```

Shell Scripting

294. What is shell scripting?

Answer: Writing a sequence of shell commands in a script to automate tasks.

295. How do you start a Bash script?

Answer: Use a shebang such as #!/bin/bash on the first line.

296. What is a shebang?

Answer: It tells the operating system which interpreter should execute the script.

297. How do you make a shell script executable?

Answer: Use chmod +x script.sh, then run it as ./script.sh.

298. How do you define a variable in Bash?

Answer: Use name=value with no spaces around =.

299. How do you read a variable?

Answer: Use $name or ${name}.

300. Why use ${name}?

Answer: It clearly delimits the variable name and is useful when adjacent characters follow it.